

27. Linked lists

Herman Kamper

This is the start of the last big topic of the course: dynamic data structures. It has a reputation for being tricky, and students do sometimes struggle with it. I don't think it is actually difficult. What makes it feel hard is that it brings together three things you have already seen: structures, pointers and `malloc`. If you are comfortable with all three, then linked lists are just a clever way of combining them. If you are not, linked lists will expose exactly where the gaps are.

In this lecture we meet the simplest dynamic data structure, the linked list. We will see how a node is defined, how nodes are strung together, and how to search a list, insert into it and delete from it. Most of the lecture is spent stepping through pictures, because the pictures are what make the code make sense.

Motivation: lists that grow and shrink

So far we have stored a collection of things in one of two ways. Either we declare an array, or, if we are a bit fancier, we reserve a block of memory with `malloc` and treat it as an array:

```
int array[100];  
int *otherArray = malloc(numElements * sizeof(int));
```

In both cases the size is fixed once the block has been reserved. With the array, the size is known before the program even runs. With `malloc`, the size can be decided while the program runs, but once the operating system has handed you the block, you cannot make it bigger or smaller. (You can ask for a new, bigger block and copy everything across, but that is extra work that you have to do yourself.)

In many real situations this is not what you want. Imagine you are a bank. When you open your doors, you have a handful of customers; over time the list of customers grows. If you are a bad bank, you might lose customers, and then the list should shrink. Or imagine a robot moving through a maze, where you want to remember every square it has visited, and there is no limit on how many steps it can take. In both cases the number of elements changes while the program runs.

One option is to define an array that is enormous, say a million elements, and hope it is enough. But that blocks off a huge amount of memory for no reason, just in case you ever get a million

What is a node?

A linked list is built from little pieces called *nodes*, which are linked together. Each node contains two things:

1. the data that we want to store;
2. a pointer to the next node in the list.

If we want to store integers, the data is an integer. If we want to store bank customers, the data is a structure with an account number, a first name, a last name and so on. For now we will keep things as simple as possible and store a single character. A node is then defined like this:

```
struct listNode {  
    char data;  
    struct listNode *nextPtr;  
};  
typedef struct listNode ListNode;
```

This is a perfectly valid structure, even though it looks strange at first: the structure contains a pointer to another structure of its own type. It is called a *self-referential* structure. Note that it does not contain a complete `struct listNode` inside itself (that would be impossible, since it would never end); it contains only a pointer to one, and a pointer always has a known size. As usual, the typedef just saves us from writing `struct listNode` over and over: from now on,

Linking two nodes together

Let us make this concrete with a small and rather contrived example. We define two nodes and link them:

```
struct listNode {
    char data;
    struct listNode *nextPtr;
};
typedef struct listNode ListNode;

int main()
{
    ListNode firstNode = {'c', NULL};
    ListNode secondNode = {'k', NULL};

    firstNode.nextPtr = &secondNode;

    return 0;
}
```

The start pointer and the arrow notation

Every linked list needs a way for the program to find its first node. This is a pointer that we usually call `startPtr` (or just `start`), which holds the address of the first node. When the list is empty, there is no first node, so `startPtr` is `NULL`. When the first node is created, `startPtr` is made to point to it.

If we lose the start pointer, we lose the whole list, because it is the only way into it.

Here is how we would set this up for our row-and-column example:

```
ListNode *start;  
ListNode firstNode = {2, 1, NULL};  
start = &firstNode;  
ListNode secondNode = {2, 2, NULL};  
  
firstNode.nextPtr = &secondNode;
```

The first line creates a box called `start`. What does this box hold? An address, so it takes eight bytes, and the address is that of a `ListNode`. The line `start = &firstNode;` puts the address of `firstNode` in that box. If `firstNode` lives at address 16, then 16 goes into `start`, and we draw an arrow from `start` to `firstNode`.

Creating nodes while the program runs

Everything we have done so far was a bit silly, because we created every node by hand in the code. We had to know, when writing the program, how many nodes there would be and what would go into them. That is the very thing we wanted to get away from.

In a real program, such as the one for the bank, the list starts out with nothing in it. When the first customer arrives, the program creates a node for them *while it is running*. When the second customer arrives, the program creates another node, and makes sure that the first node points to it. And so on. To create a node while the program runs, we use `malloc`, which reserves enough bytes for one node; and to destroy a node that is no longer needed, we use `free`. We always use `sizeof` to say how many bytes a node needs:

```
ListNode *newPtr = malloc(sizeof(ListNode));
```

This is the real reason `malloc` exists. In the examples above we could have written `ListNode a = {7, NULL};` and then pointed at `a`, but that only works if we know when writing the program that there will be a node with a 7 in it. If instead the user types in the value while the program runs, and we create a node inside a loop, a local variable would be the same box on every pass through the loop, and each new value would overwrite the previous one instead of creating a new node. With `malloc`, each call gives you a brand new box.

With a list in hand, there are three things that we typically want to do with it: search for a value

Searching a list

Suppose that we want to know whether the value 5 is in the list above, and if so, where. The code is short:

```
value = 5;
currentPtr = startPtr;
while (currentPtr != NULL && value != currentPtr->data) {
    currentPtr = currentPtr->nextPtr;
}
```

The best way to understand this is to step through it with the picture below. Each node in the picture has an address, which we write next to it. Suppose the nodes live at addresses 9046, 114, 9011 and 604, in that order.

```
value = 5;
currentPtr = startPtr;
while ( currentPtr != NULL && value != currentPtr->data ) {
    currentPtr = currentPtr->nextPtr;
}
```

Inserting a node

Now we turn to the question of where a list comes from. The answer is that we build it one node at a time. We start with a `startPtr` that is `NULL`; then we create a node with `malloc`, make `startPtr` point at it, and so on.

Inserting a node into the middle of a list is the trickiest case, and it shows how nodes are created. Suppose we have the list 1, 4, 5, 9 and we want to insert a new node holding 7 between the 5 and the 9. We assume that an earlier search has left us with two pointers: `previousPtr`, pointing at the node with the 5, and `currentPtr`, pointing at the node with the 9. The new node must go between them.

```
newPtr = malloc(sizeof(ListNode));  
newPtr->data = 7;  
newPtr->nextPtr = NULL;  
previousPtr->nextPtr = newPtr;  
newPtr->nextPtr = currentPtr;
```

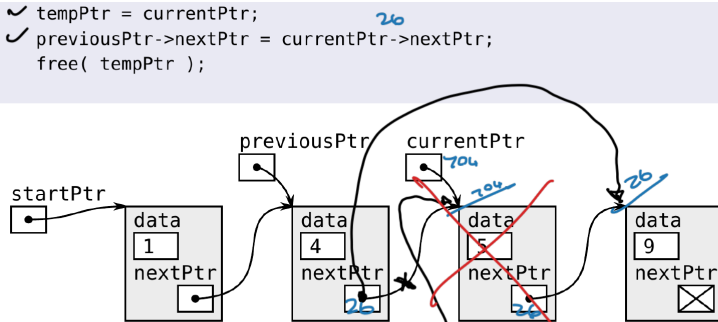
(Strictly, a 7 between a 5 and a 9 keeps the list in increasing order, which is why this position is a sensible one to pick.) The figure below shows the state of things after all of these lines have run. Let us go through them one by one.

```
newPtr = malloc( sizeof( ListNode ) );
```

Deleting a node

Deleting is the opposite of inserting. Suppose a customer has left the bank and we want to remove their node, the one holding 5, from the list 1, 4, 5, 9. We first search for it, and this time we end up with `previousPtr` pointing at the node before the one we want to delete (the 4), and `currentPtr` pointing at the node we want to delete (the 5). Then:

```
tempPtr = currentPtr;  
previousPtr->nextPtr = currentPtr->nextPtr;  
free(tempPtr);
```



Linked lists compared with arrays

Anything you can store in a linked list you can also store in an array, so why bother? The main reason is that a linked list is dynamic. Its size can change with the requirements of the program, while the size of an array is fixed.¹ A linked list is also useful when you do not know in advance how many elements you will have. An array can get full; a linked list can almost always grow to hold more data, because it only needs one more node's worth of memory at a time. If you make an array big enough to be safe, you may waste a lot of memory, or even run out of it; a linked list takes only as much as it needs.

The comparison is not all in favour of linked lists. Searching a linked list means following pointers from the start, so you cannot jump to element number 500 directly as you can with an array. Each node also stores a pointer in addition to its data, so a linked list uses more memory per element. And inserting into the middle of an array means moving all the elements after the insertion point, whereas inserting into a linked list means changing two pointers (once you have found the right position, which takes a search).

When would you insert in the middle at all? Often you would not: in the maze example, you would always add a new node at the end. Stacks and queues, which we will see next, always add and remove at one end or the other, which makes them very efficient. But if you keep a list of students sorted by student number, then a new student has to go in at the right place for the list to stay sorted, and that means inserting in the middle.

Summary

A linked list is a linear collection of nodes connected by pointers. Each node stores some data and a pointer to the next node, and a pointer such as `startPtr` identifies the first node. The `nextPtr` of the last node is `NULL`, and an empty list is one where `startPtr` is `NULL`.

The essential node definition is:

```
struct listNode {  
    char data;  
    struct listNode *nextPtr;  
};  
typedef struct listNode ListNode;
```

To search, we set a pointer to `startPtr` and keep moving it to the `nextPtr` of the node it points at, until we find the value or the pointer becomes `NULL`. To insert, we create a node with `malloc`, fill it in, and change two pointers: the previous node must point at the new node, and the new node must point at the next one. To delete, we make the previous node point at the node after the one being deleted, and then release the removed node with `free`. Insertion and deletion at the start of the list are special cases, because they change `startPtr`.

Common mistakes to look out for are:

Exercises

1. A `ListNode` stores a character and a pointer. Why can a structure contain a pointer to its own type, but not a member of its own type?
2. In the search loop, what would go wrong if the two parts of the condition were swapped, so that it read `value != currentPtr->data && currentPtr != NULL`?
3. After the search loop ends with the value found, what does `currentPtr` hold: the value, or something else? What if the value was not found?
4. Draw the list 1, 4, 5, 9 with addresses of your choosing. Step through the insertion of 7 between 5 and 9, redrawing the boxes after each line.
5. Write the code to insert a new node at the *end* of a list, assuming the list is not empty. What does your code need to find first?
6. Write the code to delete the node with a given value, including the case where it is the first node.
7. Why is it important to call `free` on a node that has been removed from a list? What happens if the program never does this, and runs for a long time?